

МИНОБРНАУКИ РОССИИ
САНКТ-ПЕТЕРБУРГСКИЙ ГОСУДАРСТВЕННЫЙ
ЭЛЕКТРОТЕХНИЧЕСКИЙ УНИВЕРСИТЕТ
«ЛЭТИ» ИМ. В.И. УЛЬЯНОВА (ЛЕНИНА)
Кафедра МО ЭВМ

ИНДИВИДУАЛЬНОЕ ДОМАШНЕЕ ЗАДАНИЕ
по дисциплине «Введение в нереляционные системы управления базами
данных»
Тема: Приложение для ведения заметок

Студенты гр. 3344

Анахин Е.Д.

Пачев Д.К.

Ханнанов А.Ф.

Преподаватель

Заславский М.М.

Санкт-Петербург

2026

ЗАДАНИЕ

Студенты

Анахин Е.Д.

Пачев Д.К.

Ханнанов А.Ф.

Группа 3344

Тема проекта: Разработка приложения для ведения заметок

Исходные данные:

Необходимо реализовать приложение для ведения заметок с использованием СУБД ArangoDB.

Содержание пояснительной записки:

“Содержание”

“Введение”

“Сценарий использования”

“Модель данных”

“Разработанное приложение”

“Заключение”

“Приложение”

Предполагаемый объем пояснительной записки:

Не менее 20 страниц.

Дата выдачи задания:

Дата сдачи реферата:

Дата защиты реферата:

Студенты

Анахин Е.Д.

Пачев Д.К.

Ханнанов А.Ф.

Преподаватель

Заславский М.М.

АННОТАЦИЯ

В рамках данного курса предполагалось разработать приложение в команде на одну из тем. Была выбрана тема создания приложения для ведения заметок на СУБД ArangoDB. В ходе работы над проектом будет проведен анализ актуальности применения нереляционных баз данных вместо привычных SQL решений. Будет получен опыт проектирования и внедрения нереляционной базы данных в веб-приложение. Найти исходный код и всю дополнительную информацию можно по ссылке: <https://github.com/moevm/nsql1h26-notes>

SUMMARY

As part of this course, it was planned to develop an application in a team on one of the topics. The topic chosen was to create a note-taking application on the ArangoDB DBMS. During the project, the relevance of using non-relational databases instead of the usual SQL solutions will be analyzed. The experience of designing and implementing a non-relational database in a web application will be gained. The source code and all additional information can be found at the link: <https://github.com/moevm/nsql1h26-notes>

СОДЕРЖАНИЕ

	Введение	6
1	Сценарии использования	7
1.1.	Макет UI	7
1.2.	Сценарии использования для задачи	8
1.3.	Сравнение количества операций чтения/записи	9
2.	Модель данных	10
2.1.	Нереляционная модель данных	10
2.2.	Аналог модели данных для SQL СУБД	16
2.3.	Сравнение моделей	23
3.	Разработанное приложение	25
3.1	Краткое описание	25
3.2	Используемые технологии	25
3.3	Снимки экрана приложения	25
	Заключение	29
	Список использованных источников	30
	Приложение А. Документация по сборке и развёртыванию	31

ВВЕДЕНИЕ

Актуальность решаемой проблемы

У пользователей всё чаще возникает необходимость хранить большое количество информации в удобных системах для хранения, поиска и структурирования заметок.

Постановка задачи

Цель работы – разработать веб-приложение для ведения заметок с поддержкой создания, редактирования, поиска и связывания заметок между собой. Помимо этого, необходимо реализовать дополнительный функционал, такой как: предоставление другим пользователям доступа к заметкам, массовый импорт/экспорт данных, ведение многокритериальной статистики.

Предлагаемое решение

Разработано клиент-серверное приложение с использованием FastAPI, React и ArangoDB.

Качественные требования к решению

Приложение должно обеспечивать удобный пользовательский интерфейс, поиск и отображение данных, импорт/экспорт данных, расширяемость, поддержку связей между заметками.

1. СЦЕНАРИИ ИСПОЛЬЗОВАНИЯ

1.1. Макет UI

Макет пользовательского интерфейса представлен на рис. 1. Макет описывает примерную структуру страниц приложения и переходы между ними. Дизайн макета не соответствует будущему дизайну приложения и играет ознакомительную роль.



1.2. Сценарии использования для задачи

Регистрация и авторизация

Неавторизованный пользователь при попытке открыть защищённые разделы приложения перенаправляется на страницу входа.

Пользователь может зарегистрироваться, указав логин и пароль. Система проверяет корректность введённых данных и создаёт новую учётную запись. После регистрации пользователь может выполнить вход в систему. При успешной авторизации создаётся пользовательская сессия и открывается рабочее пространство заметок. Также предусмотрен выход из системы с завершением текущей сессии.

Работа с заметками

После входа пользователь получает доступ к рабочему пространству, состоящему из файловой структуры заметок, редактора и панели тегов и логов.

Пользователь может:

- создавать, редактировать и удалять заметки;
- создавать папки для организации заметок;
- выполнять поиск и фильтрацию по тексту, тегам и датам;
- добавлять теги;
- создавать ссылки между заметками.

Все изменения автоматически сохраняются в базе данных и отображаются в журнале изменений.

Управление доступом

По умолчанию заметки являются приватными. Пользователь может создать публичную ссылку на заметку и выбрать уровень доступа:

- просмотр;
- редактирование.

При переходе по ссылке система проверяет её валидность и предоставляет доступ в соответствии с установленными правами.

Работа с логами

Система ведёт журнал действий пользователей и изменений заметок. Пользователь может просматривать:

- историю изменений заметок;
- действия, связанные с аккаунтом;
- подробную информацию о конкретных изменениях.

Также поддерживаются поиск и фильтрация логов по различным параметрам.

Импорт и экспорт данных

Для администратора реализованы массовый импорт и экспорт данных всей базы.

Статистика

Система предоставляет страницу статистики, на которой пользователь может строить графики по заметкам с использованием фильтрации по тегам, папкам и датам. Это позволяет анализировать структуру и активность данных в системе.

1.3. Сравнение количества операций чтения/записи

Для разработанного приложения преобладающими являются операции чтения данных. Пользователи значительно чаще просматривают заметки, выполняют поиск, фильтрацию, открывают связанные заметки, просматривают логи и статистику, чем создают или изменяют данные. Практически любое действие в интерфейсе сопровождается чтением информации из базы данных: загрузкой списка заметок, отображением содержимого, получением тегов, истории изменений и параметров доступа.

2. МОДЕЛЬ ДАННЫХ

2.1. Нереляционная модель данных

Графическое представление модели представлено на рис. 2.

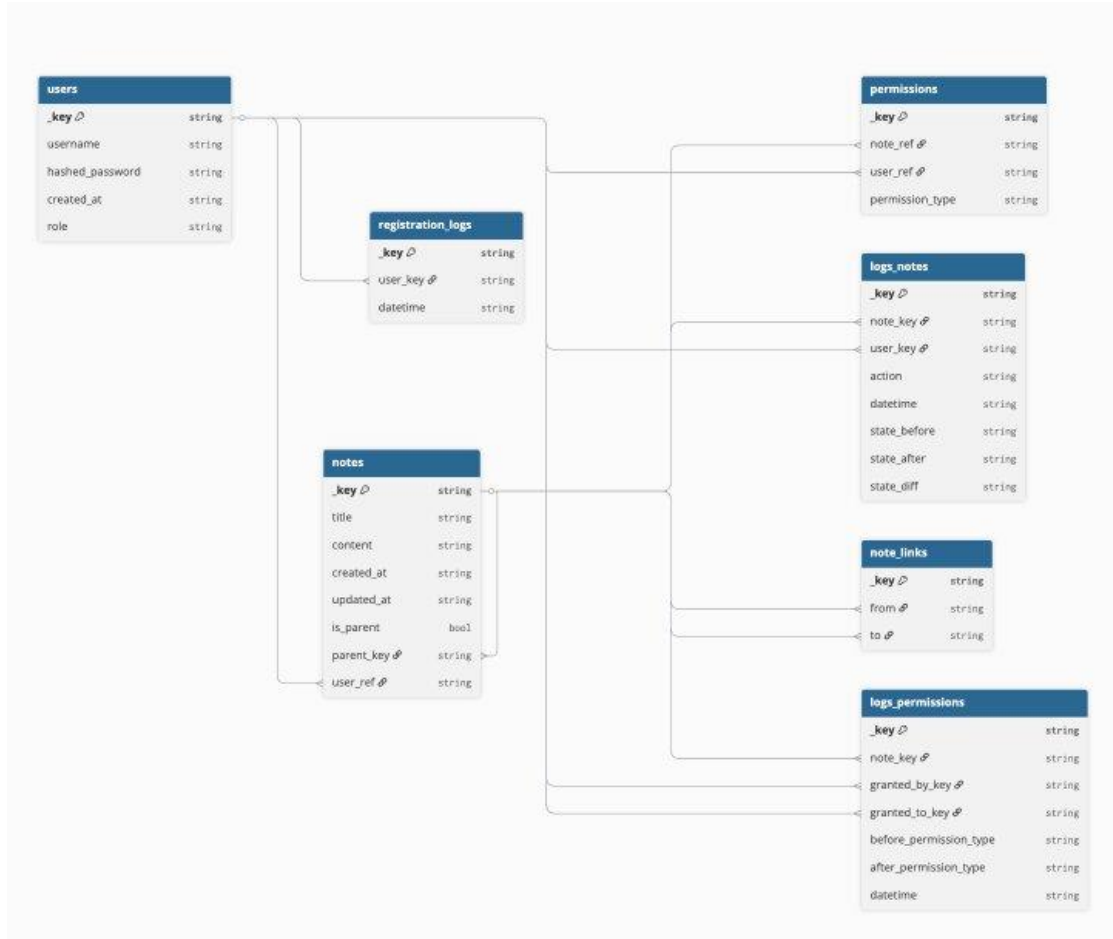


Рисунок 2 - Нереляционная модель данных

Коллекция users

Коллекция users хранит информацию о пользователях системы.

Основные поля:

- _key — уникальный идентификатор пользователя;
- username — логин пользователя;
- hashed_password — хэш пароля;
- created_at — дата регистрации;
- role — роль пользователя (user или admin).

Назначение:

- хранение учетных записей;

- авторизация и аутентификация;
- определение прав доступа.

Типы данных:

- string — идентификаторы, логин, пароль, дата, роль.

Коллекция notes

Коллекция notes является основной коллекцией приложения и хранит заметки пользователей.

Основные поля:

- _key — идентификатор заметки;
- title — заголовок заметки;
- content — текстовое содержимое;
- created_at — дата создания;
- updated_at — дата последнего изменения;
- is_parent — флаг родительской заметки;
- parent_key — ссылка на родительскую заметку;
- user_ref — ссылка на владельца заметки;
- tags — список тегов.

Назначение:

- хранение пользовательских заметок;
- организация заметок в иерархическую структуру;
- поддержка поиска и фильтрации.

Типы данных:

- string — текстовые поля и идентификаторы;
- bool — флаг родительской заметки;
- string[] — массив тегов.

Коллекция permissions

Коллекция permissions хранит информацию о правах доступа пользователей к заметкам.

Основные поля:

- `note_ref` — ссылка на заметку;
- `user_ref` — ссылка на пользователя;
- `permission_type` — тип доступа (`view` или `edit`).

Назначение:

- реализация совместного доступа к заметкам;
- разграничение прав пользователей.

Типы данных:

- `string` — идентификаторы и тип прав доступа.

Коллекция `note_links`

Коллекция `note_links` представляет графовые связи между заметками.

Основные поля:

- `_from` — исходная заметка;
- `_to` — связанная заметка.

Назначение:

- хранение связей между заметками;
- реализация навигации по связанным данным;
- построение графа знаний.

Типы данных:

- `string` — ссылки на документы коллекции `notes`.

Коллекция `logs_notes`

Коллекция `logs_notes` хранит историю изменений заметок.

Основные поля:

- `note_key` — идентификатор заметки;
- `user_key` — пользователь, выполнивший действие;
- `action` — тип действия;
- `datetime` — дата и время события;
- `state_before` — состояние до изменения;

- `state_after` — состояние после изменения;
- `diff` — различия между версиями.

Назначение:

- хранение истории изменений;
- аудит действий пользователей;
- отображение логов в интерфейсе.

Типы данных:

- `string` — идентификаторы, даты и содержимое изменений.

Коллекция `logs_permissions`

Коллекция `logs_permissions` содержит историю изменений прав доступа.

Основные поля:

- `note_key` — заметка;
- `granted_by_key` — пользователь, выдавший доступ;
- `granted_to_key` — пользователь, получивший доступ;
- `before_permission_type` — права до изменения;
- `after_permission_type` — права после изменения;
- `datetime` — дата события.

Назначение:

- контроль изменений прав доступа;
- аудит операций совместного доступа.

Типы данных:

- `string` — идентификаторы, типы прав и даты.

Коллекция `registration_logs`

Коллекция `registration_logs` хранит информацию о регистрации пользователей.

Основные поля:

- `user_key` — идентификатор пользователя;
- `datetime` — дата регистрации.

Назначение:

- хранение истории регистрации пользователей;
- аудит действий, связанных с аккаунтами.

Типы данных:

- string — идентификаторы и даты.

Для оценки объема данных были рассчитаны средние размеры документов каждой коллекции с учетом служебных полей ArangoDB (`_key`, `_id`, `_rev`).

Средний размер пользователя составляет около 191 байта. Размер заметки зависит от объема текстового содержимого и количества тегов и оценивается как $329 + C$ байт, где C — средний размер текста заметки. Размер документов прав доступа, логов и связей находится в диапазоне от 88 до 184 байт.

С учетом связанных сущностей одна заметка в среднем требует:

- хранения самой заметки;
- связанных логов изменений;
- записей о правах доступа;
- связей с другими заметками.

В результате общий объем данных для одной заметки оценивается формулой:

$$V = 1344 + 10C$$

где C — средний размер содержимого заметки.

Если количество заметок обозначить как N , то общий объем данных растет линейно:

$$V(N) = (1344 + 10C)N$$

Таким образом, увеличение числа заметок приводит к пропорциональному росту объема хранимой информации. Наиболее быстро растущими сущностями являются логи и графовые связи между заметками.

Для реализации пользовательских сценариев в приложении используются запросы на языке AQL. Основные запросы связаны с получением заметок,

управлением доступом, поиском связей между объектами и работой с журналами действий.

Работа с заметками

При открытии рабочего пространства система выполняет запрос на получение всех заметок текущего пользователя из коллекции notes:

```
FOR n IN notes
  FILTER n.user_ref == @user_key
RETURN n
```

Данный запрос используется для отображения файловой структуры и списка заметок.

При открытии конкретной заметки выполняется поиск документа по идентификатору. При редактировании заметки система обновляет поля content и updated_at.

Для поиска заметок используются запросы с фильтрацией по заголовку, содержимому, тегам и датам. В зависимости от выбранных параметров могут использоваться комбинированные фильтры.

Управление доступом

При настройке совместного доступа система обращается к коллекции permissions, содержащей информацию о правах пользователей на заметки.

Получение списка пользователей, имеющих доступ к заметке:

```
FOR p IN permissions
  FILTER p.note_ref == @note_key
  FOR u IN users
    FILTER u._key == p.user_ref
RETURN u
```

Запрос применяется при отображении текущих настроек доступа и проверке прав пользователя перед открытием заметки.

При выдаче доступа создаётся новый документ в коллекции permissions, а при изменении прав существующая запись обновляется.

Работа с логами

Для отображения истории изменений используются запросы к коллекциям logs_notes, logs_permissions и registration_logs.

При открытии истории изменений заметки система получает список логов, связанных с конкретной заметкой, сортируя их по времени изменения. Аналогично реализован просмотр пользовательских логов и логов изменения прав доступа.

Для поиска логов используются фильтры по:

- пользователю;
- типу действия;
- диапазону дат;
- заметке;
- тегам.

Проверка состояния данных

В системе также используются служебные запросы для анализа структуры данных. Например, поиск заметок, к которым не выдан доступ другим пользователям:

```
FOR n IN notes
  FILTER NOT EXISTS (
    FOR p IN permissions
      FILTER p.note_ref == n._key
      LIMIT 1
      RETURN 1
  )
RETURN n
```

Подобные запросы позволяют проверять корректность данных и использовать статистику внутри приложения.

2.2. Аналог модели данных для SQL СУБД

Графическое представление модели представлено на рис. 3.

Назначение:

- разграничение прав пользователей;
- поддержка административных функций.

Таблица notes

Таблица notes является основной таблицей системы и хранит заметки пользователей.

Основные поля:

- note_id — идентификатор заметки;
- title — заголовок;
- content — содержимое заметки;
- created_at — дата создания;
- updated_at — дата изменения;
- is_parent — признак родительской заметки;
- parent_id — ссылка на родительскую заметку;
- user_id — владелец заметки.

Назначение:

- хранение заметок;
- организация заметок в древовидную структуру;
- поддержка поиска и редактирования.

Таблица tags

Таблица tags хранит теги заметок.

Основные поля:

- tag_id — идентификатор тега;
- name — название тега.

Назначение:

- хранение тегов;
- поддержка поиска и фильтрации.

Таблица tags_to_notes

Таблица tags_to_notes реализует связь многие-ко-многим между заметками и тегами.

Основные поля:

- tag_id — ссылка на тег;
- note_id — ссылка на заметку.

Назначение:

- привязка тегов к заметкам;
- организация фильтрации по тегам.

Таблица note_permissions

Таблица note_permissions хранит права доступа пользователей к заметкам.

Основные поля:

- permission_id — идентификатор записи;
- note_id — заметка;
- user_id — пользователь;
- permission_type — тип доступа (View или Edit).

Назначение:

- реализация совместного доступа;
- проверка прав пользователей.

Таблица note_logs

Таблица note_logs хранит историю изменений заметок.

Основные поля:

- log_id — идентификатор лога;
- action — тип действия;
- created_at — дата события;
- before — состояние до изменения;
- after — состояние после изменения;
- diff — различия между версиями;
- note_id — заметка;
- editor_id — пользователь, выполнивший изменение.

Назначение:

- хранение истории изменений;
- аудит действий пользователей.

Таблица `permission_logs`

Таблица `permission_logs` содержит историю изменений прав доступа.

Основные поля:

- `log_id` — идентификатор записи;
- `permission_type` — тип доступа;
- `created_at` — дата события;
- `note_id` — заметка;
- `editor_id` — пользователь, выдавший доступ;
- `receiver_id` — пользователь, получивший доступ.

Назначение:

- аудит операций совместного доступа;
- хранение истории изменения прав.

Таблица `user_logs`

Таблица `user_logs` хранит действия, связанные с пользовательскими аккаунтами.

Основные поля:

- `log_id` — идентификатор записи;
- `action` — действие;
- `created_at` — дата события;
- `user_id` — пользователь;
- `editor_id` — пользователь, выполнивший изменение.

Назначение:

- хранение истории действий пользователей;
- аудит операций с аккаунтами.

Для оценки объема данных были рассчитаны средние размеры записей таблиц с учетом идентификаторов, строковых полей и служебных данных.

Средний размер записи пользователя составляет около 322 байт. Размер заметки зависит от объема текстового содержимого и определяется формулой:

$$141 + C$$

где C — средний размер содержимого заметки.

Дополнительный объем создают:

- таблицы логов;
- таблицы связей;
- таблицы прав доступа;
- таблицы тегов.

С учетом связанных сущностей одна заметка в среднем требует:

- хранения самой заметки;
- нескольких логов изменений;
- записей о правах доступа;
- тегов и связей между ними.

Общий объем данных рассчитывается по формуле:

$$V(N) = (852 + 10C)N$$

где:

- N — количество заметок;
- C — средний размер текста заметки.

Таким образом, рост объема данных является линейным. Наибольший вклад в увеличение размера базы данных вносят текстовые поля и журналы изменений.

Для реализации пользовательских сценариев используются SQL-запросы к связанным таблицам базы данных.

Работа с заметками

При открытии рабочего пространства выполняется запрос получения заметок пользователя:

```
SELECT * FROM notes WHERE user_id = 1;
```

Данный запрос используется для отображения списка заметок и файловой структуры.

При редактировании заметки выполняется обновление содержимого и даты изменения:

```
UPDATE notes
SET content = 'Updated content',
    updated_at = CURRENT_TIMESTAMP
WHERE note_id = 1;
```

Для поиска заметок используются запросы с фильтрацией по:

- заголовку;
- содержанию;
- тегам;
- датам создания и изменения.

Работа с тегами

Добавление тегов реализуется через таблицу `tags_to_notes`, связывающую заметки и теги отношением многие-ко-многим.

При фильтрации заметок по тегам используются JOIN-запросы между таблицами `notes`, `tags_to_notes` и `tags`.

Управление доступом

Информация о правах пользователей хранится в таблице `note_permissions`.

Получение пользователей, имеющих доступ к заметке:

```
SELECT u.*
FROM note_permissions p
JOIN users u ON u.user_id = p.user_id
WHERE p.note_id = 1;
```

Подобные запросы используются при проверке прав доступа и отображении настроек совместного использования заметок.

Работа с логами

История изменений заметок хранится в таблице `note_logs`.

Получение логов заметки за период:

```
SELECT *
FROM note_logs
WHERE note_id = 1
    AND created_at BETWEEN '2026-03-01' AND '2026-03-31'
ORDER BY created_at DESC;
```

Аналогичным образом реализуются запросы к таблицам `permission_logs` и `user_logs`.

Для поиска и фильтрации логов используются:

- фильтрация по диапазону дат;
- фильтрация по типу действия;
- фильтрация по пользователю;
- сортировка по времени.

2.3. Сравнение моделей

Удельный объём информации

Для оценки эффективности хранения данных были рассчитаны средние объёмы информации, занимаемые одной заметкой с учетом связанных сущностей.

Для SQL-модели объём данных определяется формулой:

$$V_{SQL}(N) = (852 + 10C)N$$

Для NoSQL-модели:

$$V_{NoSQL}(N) = (1344 + 10C)N$$

где:

- N — количество заметок;
- C — средний размер содержимого заметки.

При среднем размере содержимого заметки $C = 500$ байт получаем (объём для одной заметки):

SQL – 5852 байт

NoSQL – 6344 байт

Таким образом, NoSQL-модель занимает примерно на 8% больше памяти. Это связано с хранением дополнительных служебных полей, денормализацией данных и использованием отдельных документов для связей и логов.

Запросы по отдельным пользовательским сценариям

Юзкейс	SQL	NoSQL	Кол-во коллекций/таблиц

Получить все заметки пользователя	JOIN notes + users	1 коллекция notes с фильтром по user_ref	SQL: 2, NoSQL: 1
Получить пользователей с доступом к заметке	JOIN note_permissions + users	AQL: permissions + users	SQL: 2, NoSQL: 2
Получить логи заметок за период	note_logs (фильтр)	logs_notes (фильтр)	1 в обеих
Добавить новую заметку	INSERT notes + FK проверки	INSERT notes	SQL: 1 (+ FK проверка), NoSQL: 1
Добавить тег к заметке	INSERT tags_to_notes + проверка	note_links или массив ссылок	SQL: 2, NoSQL: 1

Вывод

SQL-модель показывает немного меньший удельный объем информации на заметку, благодаря более компактному хранению и фиксированным полям. Однако для многих операций приходится использовать несколько таблиц и JOIN, что увеличивает сложность запросов и нагрузку на СУБД при росте числа объектов.

NoSQL-модель занимает чуть больше места (~8%), но за счёт денормализации и вложенных структур позволяет обходиться меньшим числом коллекций и упрощает добавление данных и выборки по отдельным юзкейсам.

NoSQL даёт меньшую сложность запросов, большую гибкость и лучше масштабируется под наши задачи. Дополнительные 8% места — приемлемая цена за эти преимущества, поэтому для нашего проекта NoSQL - более практичный выбор.

3. РАЗРАБОТАННОЕ ПРИЛОЖЕНИЕ

3.1. Краткое описание

Разработанное приложение представляет собой веб-приложение для ведения заметок и организации персональной базы знаний. Архитектура приложения построена по клиент-серверному принципу и разделена на frontend, backend и database слои.

Система состоит из трёх Docker-контейнеров:

- клиентская часть;
- серверная часть;
- база данных.

Frontend реализован на основе React и отвечает за пользовательский интерфейс. Backend разработан с использованием FastAPI и предоставляет REST API для работы с заметками, пользователями, логами и системой доступа. В качестве базы данных используется ArangoDB.

Для контейнеризации и запуска приложения используется Docker Compose.

3.2. Используемые технологии

TypeScript, React – клиентская часть

Python, FastAPI – серверная часть

ArangoDB – база данных

Docker, Docker Compose – контейнеризация и оркестрация

3.3. Снимки экрана приложения

Снимки приложения приведены на рис. 4 - 11.

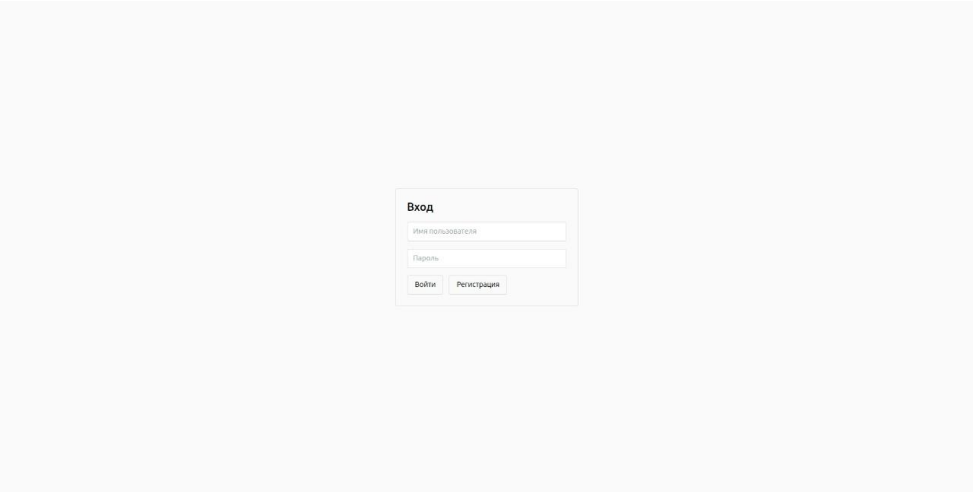


Рисунок 4 – Страница входа

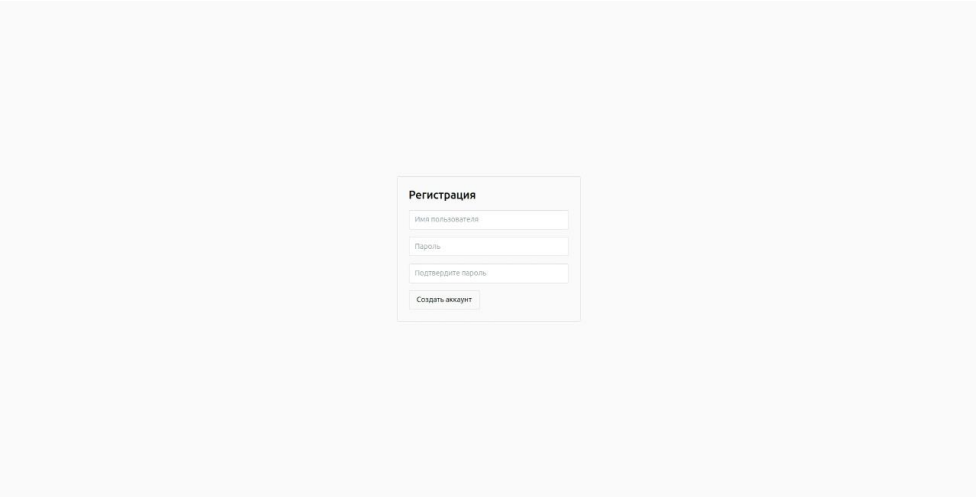


Рисунок 5 – Страница регистрации

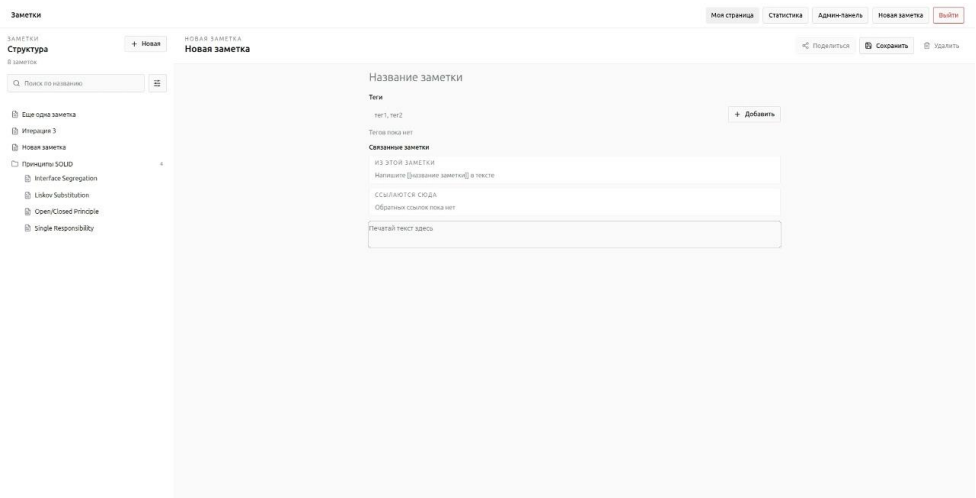


Рисунок 6 – Страница заметок

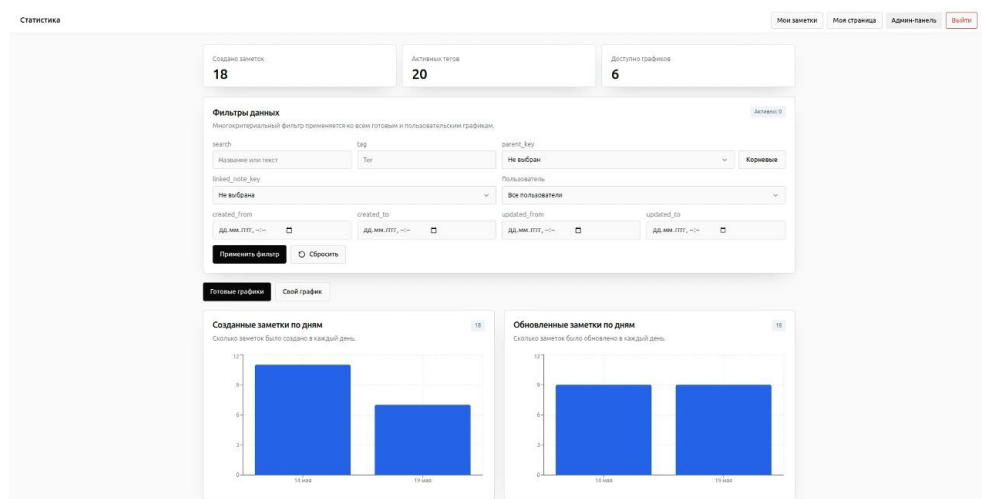


Рисунок 7 – Страница статистики

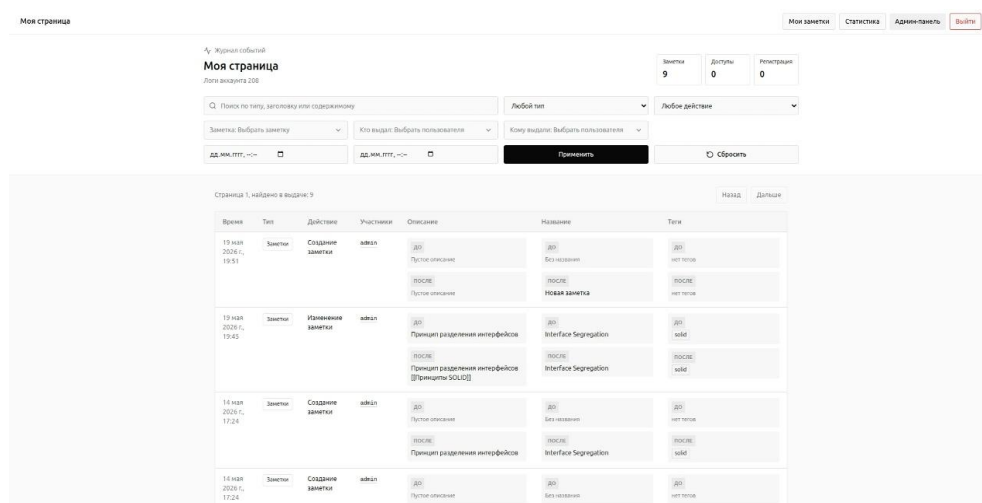


Рисунок 8 – Страница пользователя

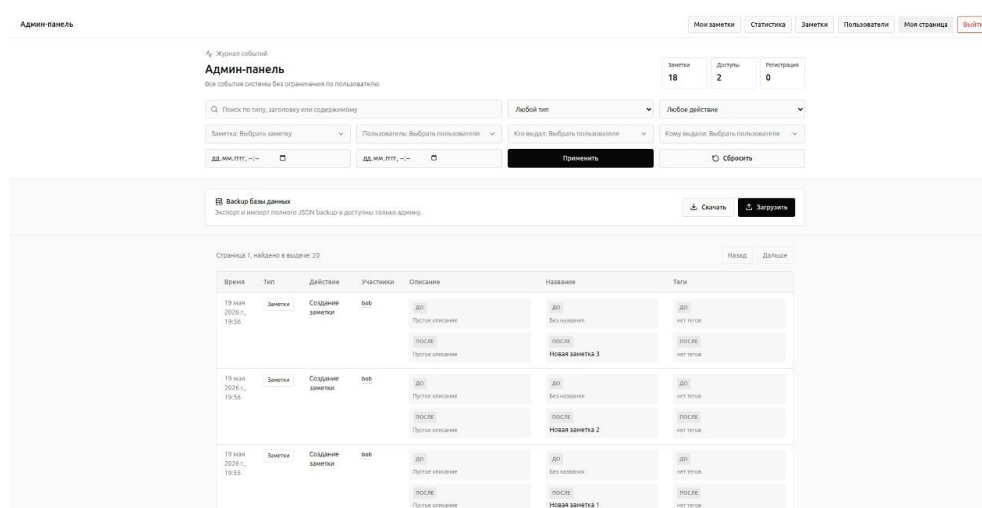


Рисунок 9 – Страница администратора

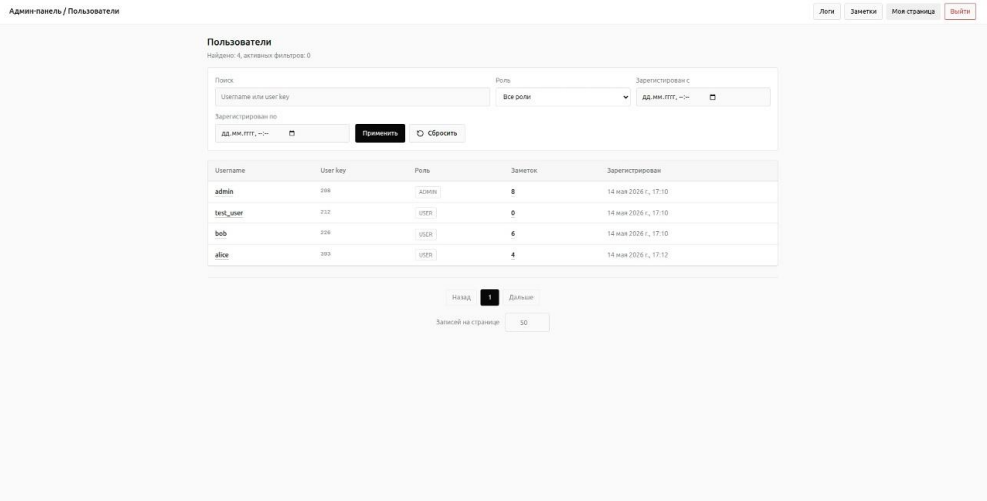


Рисунок 10 – Страница списка пользователей

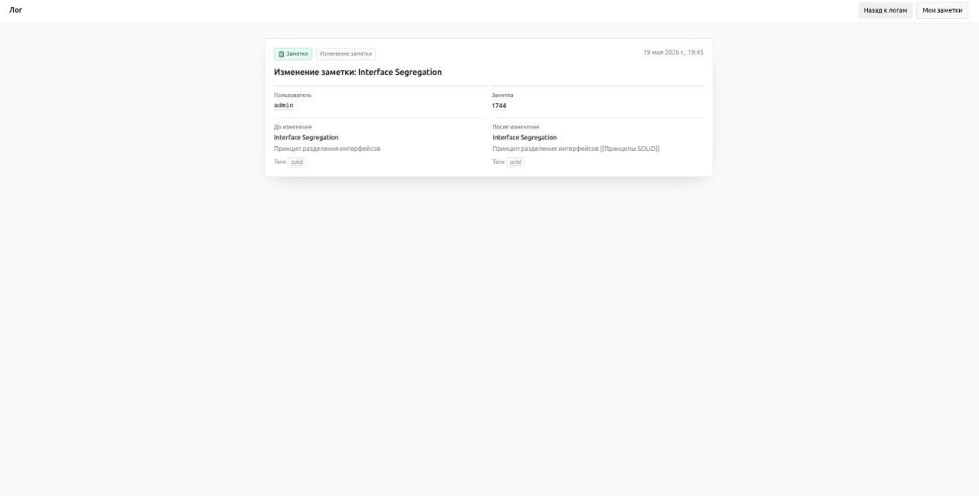


Рисунок 11 – Страница описания лога

ЗАКЛЮЧЕНИЕ

Достигнутые результаты

В рамках работы было разработано веб-приложение для ведения заметок и организации персональной базы знаний. Реализованы основные функции системы:

- регистрация и авторизация пользователей;
- создание, редактирование и удаление заметок;
- работа с тегами и связями между заметками;
- поиск и фильтрация;
- управление доступом к заметкам;
- ведение логов действий;
- массовый импорт и экспорт данных;
- построение статистики.

Недостатки и пути для улучшения полученного решения

Текущая версия приложения имеет ряд ограничений, например интерфейс приложения требует дальнейшей оптимизации, в том числе для мобильных устройств.

Для улучшения производительности приложения можно добавить кэширование и веб сокеты.

Будущее развитие решения

Благодаря грамотно написанной кодовой базе, приложение можно расширять дальше. Например, возможно добавить: более гибкую систему ролей, дополнительные функции по анализу связей заметок, ИИ ассистента.

СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ

1. Python Documentation. URL: <https://docs.python.org>
2. FastAPI Documentation. URL: <https://fastapi.tiangolo.com>
3. React Documentation. URL: <https://react.dev>
4. ArangoDB Documentation. URL: <https://docs.arango.ai/>
5. Репозиторий проекта. URL: <https://github.com/moevm/nsql1h26-notes>

ПРИЛОЖЕНИЕ А

ДОКУМЕНТАЦИЯ ПО СБОРКЕ И РАЗВЁРТЫВАНИЮ

Для запуска приложения необходимо наличие установленного Docker и Docker Compose.

Клонирование репозитория

Необходимо клонировать репозиторий проекта и перейти в директорию приложения:

```
git clone https://github.com/moevm/nsql1h26-notes.git
cd nsql1h26-notes
```

Сборка и запуск приложения

Для сборки и запуска всех контейнеров необходимо выполнить команду:

```
docker compose up -d --build
```

Проверка запуска

После успешного запуска приложение будет доступно по адресу:

```
http://localhost:4173
```

Инструкция для пользователя

После открытия приложения пользователь попадает на страницу авторизации.

Для тестирования системы можно использовать следующие учетные записи:

Роль	Логин	Пароль
Администратор	admin	adminpass
Пользователь	test_user	password
Пользователь	bob	password
Пользователь	alice	password

Основные возможности системы

После авторизации пользователь может:

- создавать и редактировать заметки
- добавлять теги

- выполнять поиск и фильтрацию
- просматривать историю изменений

Администратор дополнительно имеет доступ к:

- просмотру логов всех пользователей
- массовому импорту и экспорту данных